ENTREGA #2

AUTORES:

DANIEL FELIPE SÁNCHEZ PERAFAN
JUAN MANUEL RIVERA
NESTOR DAVID TORRES ROJAS

Presentado a:
JESUS EMILIO PINTO LOPERA
Docente

UNIVERSIDAD DE LA AMAZONIA
FACULTAD DE INGENIERÍA
PROGRAMA DE INGENIERÍA DE SISTEMAS
FLORENCIA, CAQUETÁ
2026-1

TABLA DE CONTENIDO

INTRODUCCIÓN....4
METODOLOGÍA....5
Regresión....5
Dataset de regresión y partición de los datos....5
Preprocesamiento según el algoritmo utilizado....5
Regresión lineal bayesiana....6
Máquina de soporte vectorial para regresión....7
Red neuronal artificial: las tres redes....8
Clasificación....9
Dataset de clasificación y partición de los datos....9
Preprocesamiento de los atributos....10
Naive Bayes....10
Red Bayesiana....12
Máquina de soporte vectorial....13
Red neuronal artificial: las tres redes....14
Ajuste de los umbrales y criterios de selección....15
Herramientas utilizadas....16
Métricas de evaluación....16
RESULTADOS....18
Regresión....18
Punto de partida: modelos de la primera entrega....18
Resultados de la regresión bayesiana....18
Resultados de la máquina de soporte vectorial....21
Resultados de las tres redes neuronales....23
Comparación final de los modelos....25
Evaluación sobre el mismo conjunto TEST....25
Comparación común mediante Cross-Validation....27
Costo computacional....28
Datos reales frente a predicciones....29
Clasificación....30
Punto de partida: modelos de la primera entrega....30
Resultados de Naive Bayes....30

Resultados de la Red Bayesiana.....34
Resultados de la máquina de soporte vectorial.....37
Resultados de las tres redes neuronales.....38
Comparación final de modelos.....41
CONCLUSIÓN.....46

INTRODUCCIÓN

Este informe presenta la segunda entrega del proyecto. Se trabaja con los mismos dos problemas de aprendizaje supervisado de la primera entrega: un problema de regresión, que estima el tiempo de reacción de jugadores de e-sports a partir de variables biométricas y de hábitos, y un problema de clasificación binaria, que predice si una reserva hotelera será cancelada.

En la primera entrega se compararon el Árbol de Decisión, Random Forest y KNN. Ahora se agregan tres familias de modelos: los modelos bayesianos, las Máquinas de Soporte Vectorial (SVM) y las Redes Neuronales Artificiales. En clasificación se usó Naive Bayes y, como complemento, una Red Bayesiana, que permite revisar la suposición de independencia en la que se basa Naive Bayes. En regresión Naive Bayes no es aplicable, porque la salida es continua, así que se usó el enfoque bayesiano correspondiente para regresión: la regresión lineal bayesiana. Para las redes neuronales se construyeron tres redes en cada problema: una red suave y dos redes en las que aumentan las neuronas y las capas ocultas, con el fin de observar cómo responde el modelo y si su desempeño mejora con el tamaño.

El objetivo principal es comparar resultados con la primera entrega. Por eso los modelos nuevos se entrenaron con los mismos datos, la misma partición entre entrenamiento y prueba y el mismo esquema de validación cruzada. Antes de comparar, los modelos de la primera entrega se volvieron a entrenar con sus hiperparámetros finales y se comprobó que reproducen las métricas reportadas entonces, de modo que todos los modelos se evalúan sobre el mismo conjunto TEST. Cuando un modelo nuevo tuvo que entrenarse con una muestra o con menos atributos, por costo computacional o por la naturaleza del método, se indica en su sección.

Como en la primera entrega, los hiperparámetros, los umbrales de decisión y la elección entre variantes de un mismo modelo se definieron únicamente con validación cruzada dentro de TRAIN. El conjunto TEST se usó una sola vez por modelo, en la evaluación final, para evitar que la selección se ajuste a los datos de prueba.

METODOLOGÍA

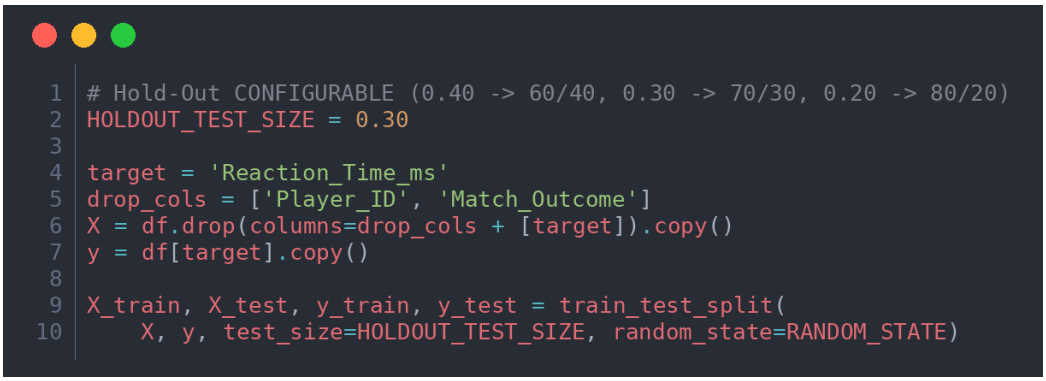
El flujo de trabajo es el mismo de la primera entrega: preparación de los datos, partición Hold-Out, preprocesamiento dentro de pipelines, selección de hiperparámetros con validación cruzada de 5 folds y evaluación final sobre TEST. La novedad es que, al final de cada problema, los modelos de las dos entregas se reúnen en una misma tabla de comparación.

Regresión

Dataset de regresión y partición de los datos

Se usó el mismo archivo data/esports_biometrics.csv, con 250 000 registros y 13 columnas. La variable objetivo es Reaction_Time_ms, el tiempo de reacción en milisegundos. Igual que antes, se excluyen Player_ID, porque es un identificador, y Match_Outcome, porque se conoce después de la partida y produciría fuga de información.

La proporción del Hold-Out se controla con el parámetro HOLDOUT_TEST_SIZE, que permite cambiar la partición a 60/40, 70/30 u 80/20 (figura 1). Con el valor 0.30 y la misma semilla (random_state = 42) se obtiene exactamente la partición de la primera entrega: 175 000 registros en TRAIN y 75 000 en TEST.



```
# Hold-Out CONFIGURABLE (0.40 -> 60/40, 0.30 -> 70/30, 0.20 -> 80/20)
HOLDOUT_TEST_SIZE = 0.30

target = 'Reaction_Time_ms'
drop_cols = ['Player_ID', 'Match_Outcome']
X = df.drop(columns=drop_cols + [target]).copy()
y = df[target].copy()

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=HOLDOUT_TEST_SIZE, random_state=RANDOM_STATE)
```

Figura 1. Partición Hold-Out configurable utilizada en la segunda entrega.

Preprocesamiento según el algoritmo utilizado

El tratamiento de faltantes y de variables categóricas no cambia: Screen_Brightness se imputa con la media, HRV_Score con la mediana, y Game_Genre y Blue_Light_Filter se codifican con One-Hot Encoding. Lo que cambia es el escalado. Los tres modelos nuevos usan el

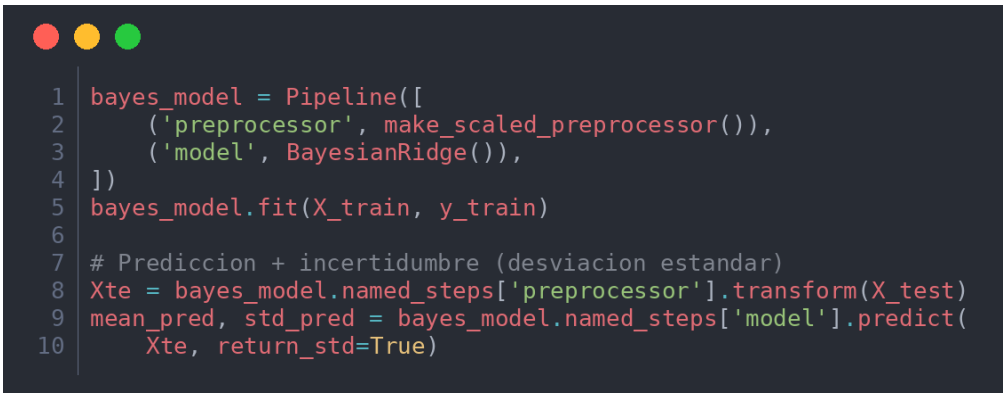
preprocesador con StandardScaler que en la primera entrega solo usaba KNN, por razones distintas en cada caso:

- Regresión bayesiana: penaliza el tamaño de los coeficientes, y si las variables tuvieran escalas distintas la penalización no las trataría por igual.
- Máquina de soporte vectorial: el kernel compara observaciones mediante productos escalares o distancias, de modo que una variable con valores grandes dominaría la comparación.
- Red neuronal: se entrena con descenso de gradiente, que converge con más dificultad cuando las entradas tienen escalas muy diferentes.

Regresión lineal bayesiana

Naive Bayes es un clasificador: estima la probabilidad de cada clase. Como aquí la salida es un número continuo, se usó el enfoque bayesiano aplicado a la regresión lineal (BayesianRidge). Una regresión lineal convencional estima un solo valor para cada coeficiente; la versión bayesiana parte de una creencia previa (prior), según la cual los coeficientes tienden a ser pequeños, y la combina con los datos mediante el teorema de Bayes. El resultado es una distribución de probabilidad para cada coeficiente y, en consecuencia, cada predicción se acompaña de una medida de incertidumbre.

El modelo no necesita una búsqueda en rejilla, porque las precisiones (inversos de la varianza) del ruido, α, y de los coeficientes, λ, se estiman a partir de los mismos datos de entrenamiento. Para compararlo con los demás modelos se evaluó con validación cruzada sobre la muestra común de 8 000 registros de TRAIN usada en la comparación por validación cruzada de la primera entrega. La figura 2 muestra cómo se entrena el modelo y cómo se obtiene la predicción junto con su desviación estándar.



```
bayes_model = Pipeline([
    ('preprocessor', make_scaled_preprocessor()),
    ('model', BayesianRidge()),
])
bayes_model.fit(X_train, y_train)

# Prediccion + incertidumbre (desviacion estandar)
Xte = bayes_model.named_steps['preprocessor'].transform(X_test)
mean_pred, std_pred = bayes_model.named_steps['model'].predict(
    Xte, return_std=True)
```

Figura 2. Regresión bayesiana: entrenamiento y predicción con incertidumbre.

Máquina de soporte vectorial para regresión

La SVR (Support Vector Regression) ajusta una función rodeada de un tubo de ± ε alrededor de la predicción (ε se expresa en milisegundos, porque la variable objetivo no se escaló). Los errores menores que ε no se penalizan; los puntos que quedan sobre el borde del tubo o fuera de él son los vectores de soporte, y son los únicos que determinan la función final. El parámetro C regula el equilibrio entre una función más plana y menos errores fuera del tubo: con un C grande el modelo se ajusta más a los datos de entrenamiento. El kernel define la forma de la función, que puede ser lineal o radial (RBF); este último permite representar relaciones no lineales.

La rejilla de kernel, C y ε se evaluó con validación cruzada de 5 folds sobre una muestra reproducible de 8 000 registros de TRAIN (figura 3); en el kernel RBF se mantuvo gamma = 'scale', el valor por defecto de scikit-learn. Se trabajó con una muestra porque el costo de entrenar una SVR con kernel crece entre el cuadrado y el cubo del número de registros, y con 175 000 registros no es viable. El entrenamiento final se definió de antemano: con el TRAIN completo mediante LinearSVR, una implementación de la SVR lineal pensada para muchos datos, si ganaba el kernel lineal, o con una muestra de 30 000 registros si ganaba el RBF.

```
svr_grid = GridSearchCV(
    Pipeline([('preprocessor', make_scaled_preprocessor()),
              ('model', SVR())]),
    param_grid={'model__kernel': ['linear', 'rbf'],
                'model__C': [1, 10, 100],
                'model__epsilon': [0.5, 2, 5]},
    scoring=scoring, refit='RMSE', cv=cv5, n_jobs=-1)
svr_grid.fit(X_svr_cv, y_svr_cv)
```

Figura 3. Código de la búsqueda de kernel, C y ε de la SVR.

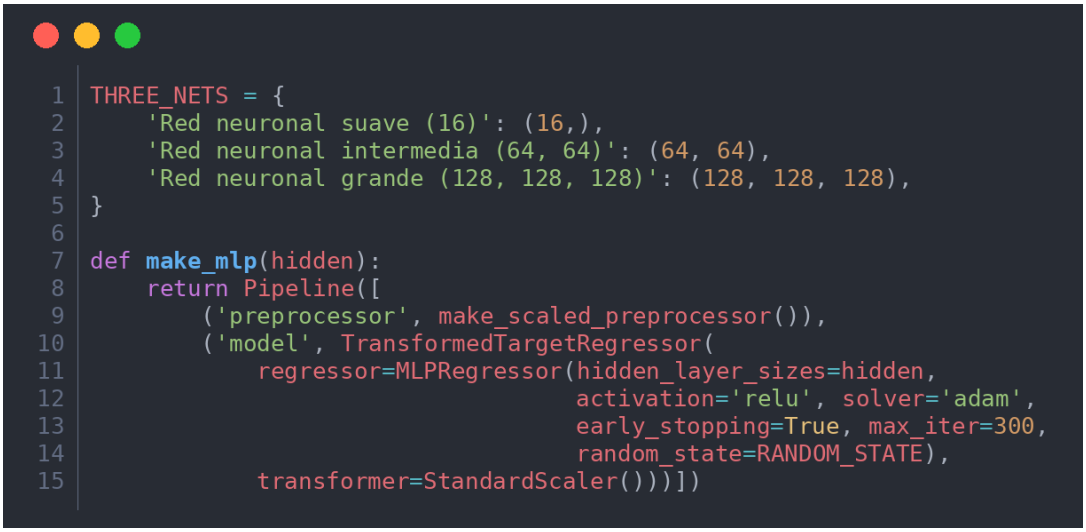
Red neuronal artificial: las tres redes

Se usó un perceptrón multicapa (MLPRegressor). Cada neurona oculta multiplica sus entradas por unos pesos, suma un sesgo y aplica la activación ReLU; la neurona de salida es lineal y entrega el tiempo de reacción estimado. Los pesos se ajustan con el optimizador Adam mediante retropropagación del error. Las tres redes se resumen en la tabla 1: de una a la siguiente aumentan a la vez las capas ocultas y las neuronas por capa.

Red	**Capas ocultas**	**Neuronas por capa**	**Arquitectura**
Suave	1	16	(16)
Intermedia	2	64	(64, 64)
Grande	3	128	(128, 128, 128)

Tabla 1. Arquitecturas de las tres redes neuronales.

Las tres redes se entrenaron en las mismas condiciones: activación ReLU, optimizador Adam y early stopping, que separa el 10 % de los datos de entrenamiento y detiene el aprendizaje cuando el R^2 en esa porción deja de mejorar durante 10 épocas. La variable objetivo se estandarizó (TransformedTargetRegressor), porque la red aprende con más facilidad valores cercanos a cero que valores alrededor de 260 ms. La regularización (alpha) y la tasa de aprendizaje se dejaron en sus valores por defecto, para que la única diferencia entre las redes fuera su tamaño. Cada red se evaluó con validación cruzada de 5 folds sobre una muestra de 20 000 registros de TRAIN y después se entrenó con el TRAIN completo para evaluarla una vez sobre TEST (figura 4).



```
THREE_NETS = {
    'Red neuronal suave (16)': (16,),
    'Red neuronal intermedia (64, 64)': (64, 64),
    'Red neuronal grande (128, 128, 128)': (128, 128, 128),
}

def make_mlp(hidden):
    return Pipeline([
        ('preprocessor', make_scaled_preprocessor()),
        ('model', TransformedTargetRegressor(
            regressor=MLPRegressor(hidden_layer_sizes=hidden,
                                   activation='relu', solver='adam',
                                   early_stopping=True, max_iter=300,
                                   random_state=RANDOM_STATE),
            transformer=StandardScaler()))])
```

Figura 4. Definición de las tres redes neuronales de regresión.

Para decidir si una red más grande mejora el resultado, cada red se compara con la anterior. Si la diferencia de RMSE de validación cruzada es menor que la desviación estándar media entre folds de las tres redes, se considera que el resultado quedó igual. Es un criterio práctico: la diferencia cabe dentro de la variación propia de la validación y no basta para preferir una red sobre otra. Además, se evaluaron las nueve combinaciones de 1, 2 y 3 capas con 16, 64 y 128 neuronas, para separar el efecto de las capas del efecto de las neuronas.

Clasificación

Dataset de clasificación y partición de los datos

Se usó el dataset de reservas hoteleras depurado en la primera entrega: 119 390 registros y 29 columnas (28 atributos y la variable objetivo), sin las variables que filtran el resultado, como reservation_status, reservation_status_date o assigned_room_type. La variable objetivo is_canceled tiene un 62.96 % de reservas no canceladas y un 37.04 % de canceladas. La partición Hold-Out estratificada 70/30, con la misma semilla, reproduce la de la primera entrega: 83 573 registros en TRAIN y 35 817 en TEST, con la misma proporción de clases.

Preprocesamiento de los atributos

Se reutilizó el preprocesamiento de la primera entrega, ajustado siempre con TRAIN. Las variables numéricas se imputan con la mediana; la tarifa diaria (adr) se recorta en su percentil 99.5 para limitar valores extremos; en country se conservan los 15 países más frecuentes y el resto se agrupa en una sola categoría; y las variables categóricas se codifican con One-Hot Encoding, lo que deja 99 columnas de entrada. La SVM y las redes neuronales usan además StandardScaler, por las mismas razones expuestas en regresión; GaussianNB comparte ese mismo preprocesador, aunque en su caso el escalado prácticamente no cambia el modelo. CategoricalNB requiere un tratamiento distinto, que se describe en su sección.

Naive Bayes

Naive Bayes aplica el teorema de Bayes para estimar la probabilidad de que una reserva se cancele, dados sus atributos:

$$P(\text{cancela} \mid x_1, \ldots, x_n) \propto P(\text{cancela}) \cdot P(x_1 \mid \text{cancela}) \cdot P(x_2 \mid \text{cancela}) \cdot \ldots \cdot P(x_n \mid \text{cancela})$$

La probabilidad de cancelar es proporcional a la tasa general de cancelación (el prior, 37 %) multiplicada por la probabilidad de observar el valor de cada atributo entre las reservas canceladas. Escribirla como un producto solo es correcto si los atributos son independientes entre sí dentro de cada clase. Esa es la suposición «ingenua» del método: se adopta porque simplifica mucho el cálculo, aunque los datos no la cumplan.

Antes de entrenar el modelo se midió qué tan lejos están los datos de esa suposición, mediante la V de Cramér entre cada par de atributos dentro de cada clase. Para ello se usó una versión discretizada de once atributos (tabla 2), entre ellos los que en la primera entrega mostraron mayor capacidad para separar las clases, como lead_time, country, total_of_special_requests y booking_changes. Es la misma versión que después utiliza la Red Bayesiana. La V se calculó por separado en las reservas canceladas y en las no canceladas, y se promedió ponderando por el tamaño de cada clase.

Nombre en la red	Variable original	Categorías
deposit_type	deposit_type	No Deposit, Non Refund, Refundable
lead_time	lead_time (días)	0-7, 8-30, 31-90, 91-180, más de 180
country	country	PRT, GBR, FRA, ESP, DEU (los cinco más frecuentes en TRAIN) y Otro
market_segment	market_segment	Categorías originales (Online TA, Offline TA/TO, Groups, etc.)
special_requests	total_of_special_requests	0, 1, 2 o más
prev_cancel	previous_cancellations	sí (al menos una), no
parking	required_car_parking_spaces	sí (al menos uno), no
customer_type	customer_type	Transient, Transient-Party, Contract, Group
booking_changes	booking_changes	sí (al menos uno), no
hotel_type	hotel_type	City Hotel, Resort Hotel
adr	adr (tarifa diaria)	≤ 70, 70-100, 100-140, > 140 (cada intervalo incluye su límite superior)

Tabla 2. Atributos discretizados usados en el análisis de dependencia y en la Red Bayesiana.

Se evaluaron dos variantes de Naive Bayes, porque el dataset combina atributos numéricos y categóricos. GaussianNB supone que cada atributo sigue una distribución normal dentro de cada clase y trabaja sobre las variables escaladas y codificadas. CategoricalNB trabaja con categorías: los atributos numéricos se discretizan en 5 o 10 intervalos por cuantiles y los categóricos se codifican como enteros (figura 5). Ambas se evaluaron con validación cruzada de 5 folds sobre todo TRAIN, y la variante se eligió por ROC-AUC. Se usó esta métrica porque mide qué tan bien ordena el modelo las reservas por riesgo sin depender de un umbral; como el umbral se ajusta después, el F1 calculado con 0.5 no es un buen criterio para escoger la variante.

```python
def make_categorical_nb(n_bins=5, alpha=1.0):
    pre = ColumnTransformer([
        ('num', Pipeline([
            ('imputer', SimpleImputer(strategy='median')),
            ('bins', KBinsDiscretizer(n_bins=n_bins, encode='ordinal',
                                      strategy='quantile', subsample=None))]),
         numeric_cols),
        ('country', Pipeline([('bucket', TopCategoryBucketer(top_n=15)),
                              ('ord', SafeOrdinalEncoder())]), ['country']),
        ('cat', SafeOrdinalEncoder(), other_cat),
    ])
    return Pipeline([('preprocessor', pre),
                     ('model', CategoricalNB(alpha=alpha, min_categories=64))])
```

Figura 5. Pipeline de Naive Bayes categórico.

Red Bayesiana

Como complemento de Naive Bayes se construyó una Red Bayesiana con la biblioteca pgmpy. Naive Bayes impone una estructura fija: la clase se conecta con todos los atributos y ningún atributo se conecta con otro. Una Red Bayesiana, en cambio, es un grafo dirigido en el que una flecha A → B indica que la distribución de B se modela condicionada a A. La estructura se aprende de los datos, de modo que el modelo puede representar las dependencias entre atributos que Naive Bayes ignora.

La estructura se obtuvo con una búsqueda Hill Climbing, que agrega, elimina o invierte flechas mientras mejore la puntuación BIC; esta puntuación premia el ajuste a los datos y penaliza los grafos con demasiados parámetros. Se permitió un máximo de tres padres por variable. Después se estimó para cada variable una tabla de probabilidades condicionales con un prior BDeu (tamaño de muestra equivalente de 10), que evita probabilidades iguales a cero (figura 6). Las categorías posibles de cada variable se tomaron solo de TRAIN.

```python
def fit_bayesian_network(train_df):
    # 1) Estructura: busqueda Hill Climbing con puntuacion BIC
    dag = HillClimbSearch(train_df).estimate(scoring_method='bic-d',
                                             max_indegree=3)
    model = DiscreteBayesianNetwork(dag.edges())
    model.add_nodes_from(train_df.columns)
    # 2) Parametros: tablas de probabilidad condicional (prior BDeu);
    #    las categorias posibles se toman solo de TRAIN
    states = {c: sorted(set(bn_train[c])) for c in train_df.columns}
    cpds = BayesianEstimator(model, train_df, state_names=states)\
        .get_parameters(prior_type='BDeu', equivalent_sample_size=10)
    model.add_cpds(*cpds)
    return model
```

Figura 6. Código del aprendizaje de la estructura y de los parámetros de la Red Bayesiana.

Una Red Bayesiana discreta necesita variables categóricas, por lo que se usaron los once atributos de la tabla 2. Los cortes de lead_time y adr se definieron con valores redondos que separan rangos con sentido práctico (una semana, un mes, un trimestre y un semestre de anticipación; tarifas bajas, medias, altas y muy altas), y las variables de conteo se resumieron en si el evento ocurrió o no, para que las tablas de probabilidad no tuvieran demasiadas combinaciones. Para que la comparación con Naive Bayes fuera justa, también se entrenó un Naive Bayes con estos mismos once atributos.

Máquina de soporte vectorial

La SVM busca la frontera que separa las reservas canceladas de las no canceladas con el mayor margen posible. Como las clases se solapan, se usa un margen suave: se permite que algunos puntos queden dentro del margen o del lado equivocado de la frontera, con una penalización controlada por C. Los vectores de soporte son los puntos que quedan sobre el margen, dentro de él o mal clasificados, y son los únicos que definen la frontera. Se evaluaron el kernel lineal, con LinearSVC y C de 0.01, 0.1 y 1, y el kernel RBF, con SVC, gamma = 'scale' y C de 0.1, 1 y 10; cada combinación se probó sin ponderación de clases y con class_weight = 'balanced', el mismo tratamiento del desbalance de la primera entrega.

La búsqueda se hizo con validación cruzada de 5 folds sobre una muestra estratificada de 8 000 registros de TRAIN (figura 7). Se definió de antemano que, si ganaba el kernel RBF, el

modelo final se entrenaría con 30 000 registros de TRAIN, y si ganaba el lineal, con el TRAIN completo mediante LinearSVC. La SVM no entrega probabilidades, sino una puntuación con signo (decision_function) proporcional a la distancia de cada reserva a la frontera; el umbral de decisión se ajustó sobre esa puntuación.

```
svm_grid = GridSearchCV(svm_pipe, param_grid=[
    {'model': [LinearSVC(dual='auto', max_iter=10000)],
     'model__C': [0.01, 0.1, 1],
     'model__class_weight': [None, 'balanced']},
    {'model': [SVC(kernel='rbf', gamma='scale')],
     'model__C': [0.1, 1, 10],
     'model__class_weight': [None, 'balanced']},
], scoring=SCORING, refit='f1', cv=cv5, n_jobs=-1)
svm_grid.fit(X_svm_cv, y_svm_cv)
```

Figura 7. Código de la búsqueda de kernel, C y class_weight de la SVM.

Red neuronal artificial: las tres redes

Se usaron las tres arquitecturas de la tabla 1 con MLPClassifier, en las mismas condiciones que en regresión. En este caso la neurona de salida usa la función logística y entrega la probabilidad de cancelación, y el early stopping vigila la exactitud (accuracy) sobre el 10 % reservado. Cada red se evaluó con validación cruzada de 5 folds sobre una muestra estratificada de 20 000 registros de TRAIN, que también se usó para elegir su umbral; después se entrenó con el TRAIN completo y se evaluó una vez sobre TEST (figura 8). También se evaluaron las nueve combinaciones de capas y neuronas.

```
for name, hidden in THREE_NETS.items():
    s = cross_validate(make_mlp(hidden), X_mlp_cv, y_mlp_cv,
                       cv=cv5, scoring=SCORING)
    oof = cross_val_predict(make_mlp(hidden), X_mlp_cv, y_mlp_cv,
                            cv=cv5, method='predict_proba')[:, 1]
    thr, _ = best_threshold(y_mlp_cv, oof, THRESHOLD_GRID)
    model = make_mlp(hidden).fit(X_train, y_train)
    scores = model.predict_proba(X_test)[:, 1]
    final_rows.append(evaluate(name, 'Entrega 2', scores, thr, ...))
```

Figura 8. Código de la evaluación de las tres redes neuronales de clasificación.

Ajuste de los umbrales y criterios de selección

Se mantuvo el criterio de la primera entrega: el umbral de cada modelo es el que maximiza F1 sobre predicciones out-of-fold, es decir, predicciones de validación cruzada en las que cada registro es evaluado por un modelo que no lo vio durante el entrenamiento. Los umbrales candidatos van de 0.10 a 0.90 en pasos de 0.01. Como el costo de cada modelo es distinto, el conjunto sobre el que se hizo el ajuste también varía:

- Naive Bayes: todo TRAIN (83 573 registros).
- Red Bayesiana y Naive Bayes de once atributos: todo TRAIN; en cada fold se volvieron a aprender la estructura y las tablas de probabilidad.
- Redes neuronales: la muestra de 20 000 registros de su validación cruzada. Los modelos de cada fold se entrenaron con 16 000 registros y el umbral se aplicó a la red entrenada con todo TRAIN, así que también es una aproximación.
- SVM: la muestra de 8 000 registros de la búsqueda, con 181 candidatos tomados de los cuantiles de su puntuación, que no está en la escala de 0 a 1. Los modelos de cada fold se entrenaron con 6 400 registros y el umbral se aplicó al modelo final entrenado con 30 000, por lo que se trata de una aproximación.
- Modelos de la primera entrega: se conservaron los umbrales elegidos entonces (0.49 para Random Forest, 0.58 para el árbol y 0.43 para KNN).

ROC-AUC y PR-AUC se calculan con las probabilidades o puntuaciones, sin aplicar umbral. En las tablas de validación cruzada, F1, Recall y Precision se calculan con el umbral por defecto (probabilidad de 0.5, o puntuación de 0 en la SVM) y se marcan con un asterisco, porque el umbral ajustado solo interviene en la evaluación final. Por último, para elegir entre las tres redes neuronales se usó la validación cruzada y no TEST; cuando las diferencias caen dentro de la variación entre folds, se prefiere la red más simple.

Herramientas utilizadas

El proyecto se desarrolló en Python con Jupyter Notebook. Se usaron Pandas para cargar y transformar los datos, NumPy para las operaciones numéricas, Matplotlib para las gráficas y NetworkX para dibujar el grafo de la Red Bayesiana. Scikit-learn siguió siendo la biblioteca principal: train_test_split, KFold, StratifiedKFold, cross_validate, cross_val_predict y GridSearchCV para la partición, la validación cruzada y la búsqueda de hiperparámetros; Pipeline, ColumnTransformer, SimpleImputer, OneHotEncoder, StandardScaler y KBinsDiscretizer para el preprocesamiento.

Los modelos nuevos fueron BayesianRidge, GaussianNB y CategoricalNB (modelos bayesianos), SVR, LinearSVR, SVC y LinearSVC (máquinas de soporte vectorial), y MLPRegressor y MLPClassifier (redes neuronales). La Red Bayesiana se construyó con pgmpy, mediante HillClimbSearch, BayesianEstimator y VariableElimination. Para la comparación se reutilizaron los modelos de la primera entrega (DecisionTree, RandomForest y KNeighbors) y los baselines (DummyRegressor y DummyClassifier). Las semillas aleatorias se fijaron en todo el proceso, de modo que los resultados se pueden reproducir.

Métricas de evaluación

Se mantuvieron las métricas de la primera entrega. En regresión se usaron MAE y RMSE (en ms), MSE (en ms^2) y R^2, con el RMSE como métrica principal. En clasificación, con la cancelación como clase positiva, se usaron Accuracy, Balanced Accuracy, Precision, Recall, F1, ROC-AUC y PR-AUC (calculada como Average Precision), con F1 como métrica principal.

Se añadieron tres medidas. La primera es el número de conexiones de cada red neuronal, es decir, la cantidad de pesos entre neuronas sin contar los sesgos, que permite comparar su tamaño. La segunda es la V de Cramér, que mide la asociación entre dos variables categóricas en una escala de 0 (independientes) a 1 (asociación perfecta) y se usó para revisar la suposición de independencia de Naive Bayes. La tercera es el tiempo de entrenamiento y de predicción de cada

modelo, medido en la misma máquina; estos tiempos son orientativos, porque varían con la carga del equipo.

RESULTADOS

Regresión

Punto de partida: modelos de la primera entrega

Se volvieron a entrenar el baseline y los tres modelos de la primera entrega con sus hiperparámetros finales. La tabla 3 muestra que reproducen exactamente el RMSE reportado entonces, lo que confirma que la partición y el preprocesamiento son los mismos.

Modelo	**RMSE TEST (ms)**	**R^2 TEST**	**RMSE reportado en la Entrega 1**	**¿Coincide?**
Árbol podado CV	15.3323	0.3502	15.3323	Sí
Random Forest podado CV	15.1495	0.3656	15.1495	Sí
KNN K=50 (distance)	15.2923	0.3536	15.2923	Sí
Baseline (media)	19.0207	0.0000	19.0207	Sí

Tabla 3. Verificación de los modelos de la primera entrega sobre TEST (regresión).

Resultados de la regresión bayesiana

En la validación cruzada sobre la muestra común, la regresión bayesiana obtuvo un RMSE de 15.0483 ± 0.3281 ms y un R^2 de 0.3886. En TEST, entrenada con los 175 000 registros, alcanzó un RMSE de 15.0609 ms y un R^2 de 0.3730, un error menor que el de los tres modelos de la primera entrega. La desviación del ruido estimada por el modelo fue de 15.07 ms, muy cercana a su RMSE, como es de esperar, porque ese parámetro se estima a partir de los residuos del ajuste.

Los coeficientes del modelo se pueden leer directamente (tabla 4 y figura 9). Como las variables numéricas están estandarizadas, su coeficiente indica cuántos milisegundos cambia la predicción cuando la variable aumenta una desviación estándar y las demás se mantienen fijas. Las columnas creadas por One-Hot Encoding no se escalaron, así que su coeficiente indica directamente la diferencia en milisegundos asociada a esa categoría.

Variable	Coeficiente (ms por desv. est.; ms en columnas One-Hot)	Incertidumbre (± 1 desviación)
Sleep_Hours	−7.471	0.037
PlayTime_Hours	7.166	0.046
Age	5.720	0.036
Energy_Drinks	−0.354	0.042
Game_Genre_Battle Royale	0.106	1.712
Blue_Light_Filter_Yes	0.092	0.075
Game_Genre_FPS	−0.086	1.710
Game_Genre_Fighting	−0.052	1.713
HRV_Score	0.041	0.046
Game_Genre_MOBA	0.033	1.710
Screen_Brightness	−0.019	0.036
Heart_Rate_BPM	0.008	0.044
APM	−0.005	0.067

Tabla 4. Coeficientes de la regresión bayesiana y su incertidumbre.

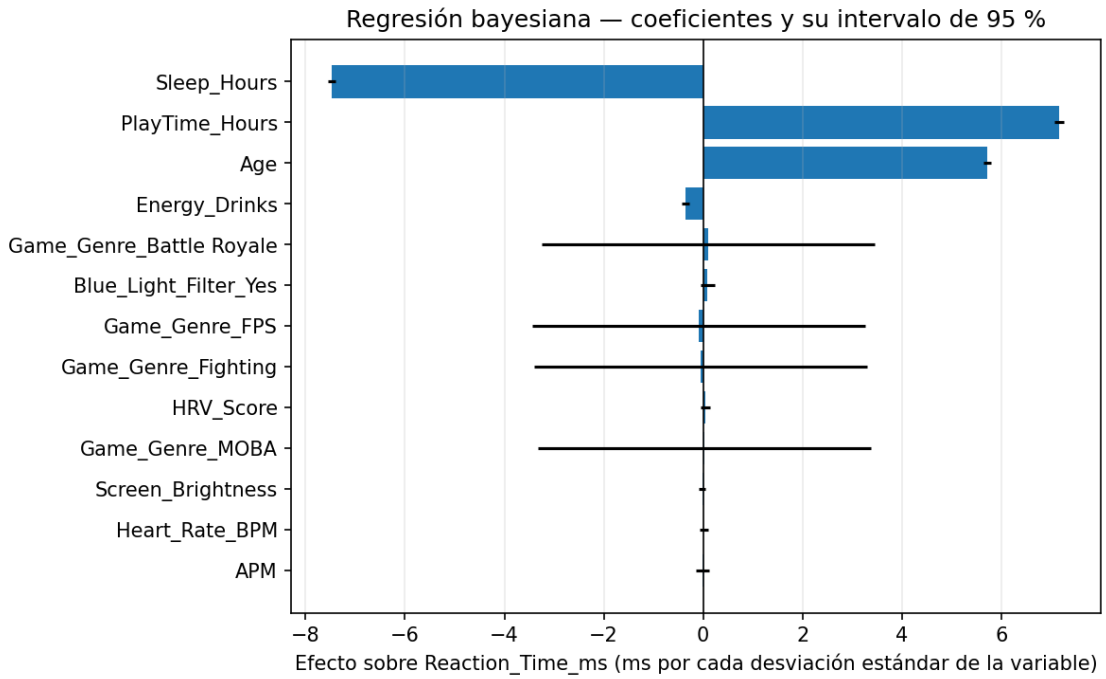


Figura 9. Coeficientes de la regresión bayesiana con su intervalo de 95 %.

Sleep_Hours, PlayTime_Hours y Age concentran casi todo el efecto: una desviación estándar más de sueño se asocia con 7.47 ms menos de tiempo de reacción, mientras que una desviación estándar más de horas de juego o de edad se asocia con 7.17 ms y 5.72 ms más, respectivamente. Energy_Drinks tiene un efecto pequeño (−0.35 ms) y el resto de variables, un efecto prácticamente nulo. Son asociaciones observadas en los datos, no efectos causales. Las tres variables principales coinciden con las que mostraron mayor capacidad predictiva en la primera entrega. HRV_Score, que en la primera entrega mostró una capacidad predictiva individual apreciable y tiene una correlación de −0.295 con el tiempo de reacción, obtiene un coeficiente casi nulo cuando el modelo incluye las demás variables, lo que sugiere que la información que aporta ya está contenida en ellas.

Los coeficientes de Game_Genre tienen una incertidumbre (± 1.71 ms) mucho mayor que la del resto. Esto no indica que el género sea una variable dudosa, sino que se debe a la codificación: se creó una columna para cada uno de los cuatro géneros, y como siempre suman 1, resultan redundantes con el intercepto. Por eso sus coeficientes individuales no quedan bien determinados, aunque esto no afecta a las predicciones.

Cada predicción incluye, además, un intervalo de 95 %. La tabla 5 muestra ocho jugadores de TEST con su valor real, la predicción y el intervalo. El intervalo es prácticamente el mismo para todos (± 29.72 ms) porque casi toda la incertidumbre proviene del ruido; la incertidumbre de los coeficientes, estimados con 175 000 registros, aporta muy poco. En todo el conjunto TEST el valor real cae dentro del intervalo en el 95.1 % de los casos, una cobertura coherente con el 95 % nominal.

Jugador	Real (ms)	Predicho (ms)	Intervalo 95 % (ms)	Error (ms)	¿Real dentro del intervalo?
1	230.83	256.39	± 29.72	−25.56	Sí
2	283.01	265.84	± 29.72	17.17	Sí
3	251.81	270.49	± 29.72	−18.68	Sí
4	253.91	275.42	± 29.72	−21.51	Sí
5	258.00	273.91	± 29.72	−15.91	Sí
6	284.60	274.34	± 29.72	10.26	Sí
7	277.94	270.35	± 29.72	7.59	Sí
8	226.66	266.19	± 29.72	−39.53	No

Tabla 5. Ejemplos de predicción con intervalo de incertidumbre (regresión bayesiana).

Resultados de la máquina de soporte vectorial

La tabla 6 presenta las 18 configuraciones evaluadas, ordenadas por RMSE de validación cruzada, y la figura 10 las muestra separadas por kernel. Las nueve configuraciones con kernel lineal ocupan los primeros lugares, con un RMSE cercano a 15.12 ms que prácticamente no cambia con C ni con ε. La mejor configuración RBF (C = 10, ε = 5) obtuvo 15.3369 ms, y con C = 100 el error subió hasta 16.1208 ms, un posible indicio de sobreajuste. Los valores elegidos para el kernel lineal quedan en el borde de la rejilla (el menor C y el mayor ε), pero como el RMSE es prácticamente plano frente a ambos parámetros, ampliar la rejilla no cambiaría el resultado de forma apreciable.

Kernel	C	ε	RMSE CV (ms)	Desv. RMSE CV	R^2 CV
lineal	1	5.0	15.1201	0.2363	0.3752
lineal	10	5.0	15.1203	0.2362	0.3752
lineal	100	5.0	15.1203	0.2361	0.3752
lineal	1	0.5	15.1214	0.2401	0.3751
lineal	100	0.5	15.1220	0.2399	0.3750
lineal	10	0.5	15.1222	0.2401	0.3750
lineal	1	2.0	15.1239	0.2411	0.3749
lineal	10	2.0	15.1252	0.2412	0.3748

Kernel	C	ε	RMSE CV (ms)	Desv. RMSE CV	R^2 CV
lineal	100	2.0	15.1253	0.2413	0.3748
RBF	10	5.0	15.3369	0.2215	0.3572
RBF	10	2.0	15.3655	0.2216	0.3548
RBF	10	0.5	15.3717	0.2159	0.3543
RBF	1	2.0	15.3798	0.2180	0.3537
RBF	1	0.5	15.3810	0.2201	0.3536
RBF	1	5.0	15.4006	0.2122	0.3520
RBF	100	5.0	15.9965	0.2616	0.3008
RBF	100	2.0	16.0642	0.2535	0.2949
RBF	100	0.5	16.1208	0.2609	0.2899

Tabla 6. Configuraciones de la SVR evaluadas mediante validación cruzada sobre 8 000 registros de TRAIN.

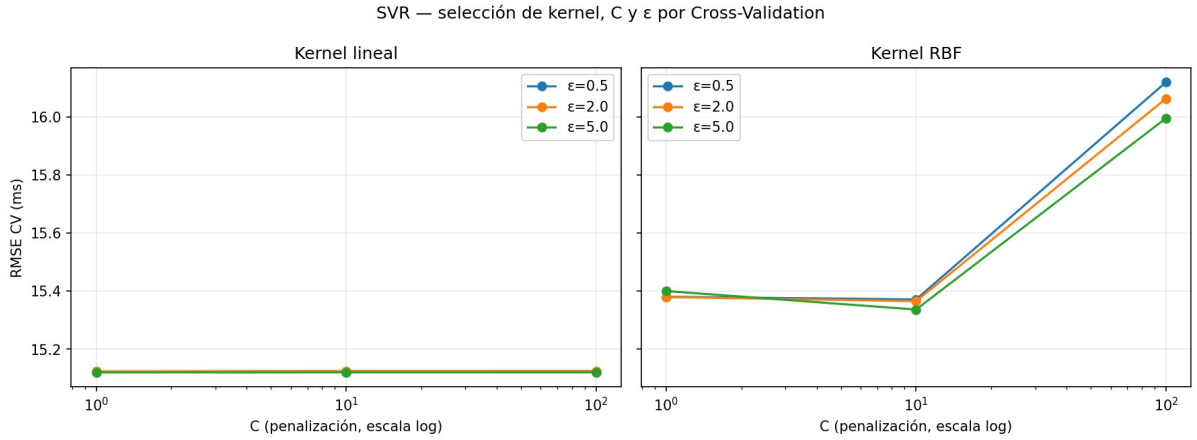


Figura 10. Selección de kernel, C y ε de la SVR mediante validación cruzada.

Se eligió el kernel lineal con C = 1 y ε = 5 (RMSE CV de 15.1201 ms). El modelo final se entrenó con LinearSVR sobre los 175 000 registros de TRAIN y obtuvo en TEST un RMSE de 15.0617 ms y un R^2 de 0.3729, prácticamente lo mismo que la regresión bayesiana. Que el kernel lineal supere al RBF es coherente con una relación aproximadamente lineal entre las variables y el tiempo de reacción, aunque la comparación tiene un límite: el kernel RBF se evaluó con 8 000 registros y sin ajustar gamma.

Resultados de las tres redes neuronales

La tabla 7 resume las tres redes. De la red suave a la grande, las conexiones pasan de 224 a 34 560, unas 154 veces más, y el tiempo de entrenamiento con el TRAIN completo pasa de 3.4 a 31.2 segundos. El error, en cambio, prácticamente no cambia: el RMSE de validación cruzada se mantiene en 15.13 ms y el de TEST en 15.00 ms en las tres redes. Las diferencias son de milésimas de milisegundo, muy por debajo de la variación entre folds (alrededor de ± 0.09 ms).

Red	**Capas ocultas**	**Neuronas por capa**	**Conexiones**	**RMSE CV (ms)**	**RMSE TEST (ms)**	**R^2 TEST**	**Respecto a la red anterior**
Suave (16)	1	16	224	15.131 ± 0.090	14.996	0.3784	Punto de partida
Intermedia (64, 64)	2	64	4 992	15.132 ± 0.082	14.998	0.3782	Quedó igual (+0.001)
Grande (128, 128, 128)	3	128	34 560	15.133 ± 0.099	14.996	0.3784	Quedó igual (+0.001)

Tabla 7. Comparación de las tres redes neuronales de regresión (CV sobre 20 000 registros de TRAIN; TEST con la red entrenada en todo TRAIN).

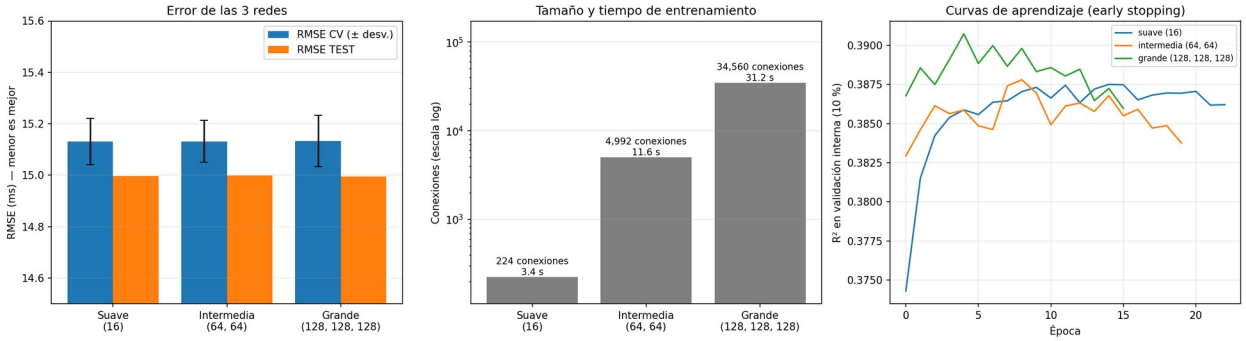


Figura 11. Error, tamaño, tiempo de entrenamiento y curvas de aprendizaje de las tres redes de regresión.

En otras palabras, aumentar neuronas y capas ocultas no cambió el error; solo aumentó el costo. Las curvas de aprendizaje de la figura 11 muestran que las redes más grandes llegan antes a su mejor punto y que el early stopping las detiene en menos épocas (16 la grande frente a 23 la suave). Según el criterio definido en la metodología, se elige la red suave, que obtiene el mismo resultado con una fracción del costo.

Las nueve combinaciones de capas y neuronas (tabla 8 y figura 12) confirman lo anterior: todas quedan entre 15.127 y 15.176 ms de RMSE CV, y ni agregar una capa ni aumentar las neuronas produce un cambio mayor que la variación de la validación cruzada. Si la relación es aproximadamente lineal, una red más grande no tiene patrones adicionales que aprovechar.

Arquitectura	**Conexiones**	**RMSE CV (ms)**	**Al agregar una capa**	**Al aumentar neuronas**
(16)	224	15.131 ± 0.090	—	—
(64)	896	15.127 ± 0.087	—	Quedó igual (−0.004)
(128)	1 792	15.127 ± 0.047	—	Quedó igual (+0.000)
(16, 16)	480	15.176 ± 0.109	Quedó igual (+0.045)	—
(64, 64)	4 992	15.132 ± 0.082	Quedó igual (+0.005)	Quedó igual (−0.044)
(128, 128)	18 176	15.152 ± 0.087	Quedó igual (+0.025)	Quedó igual (+0.020)
(16, 16, 16)	736	15.160 ± 0.075	Quedó igual (−0.015)	—
(64, 64, 64)	9 088	15.159 ± 0.088	Quedó igual (+0.027)	Quedó igual (−0.001)
(128, 128, 128)	34 560	15.133 ± 0.099	Quedó igual (−0.019)	Quedó igual (−0.026)

Tabla 8. Efecto de agregar capas y neuronas en las nueve combinaciones (regresión).

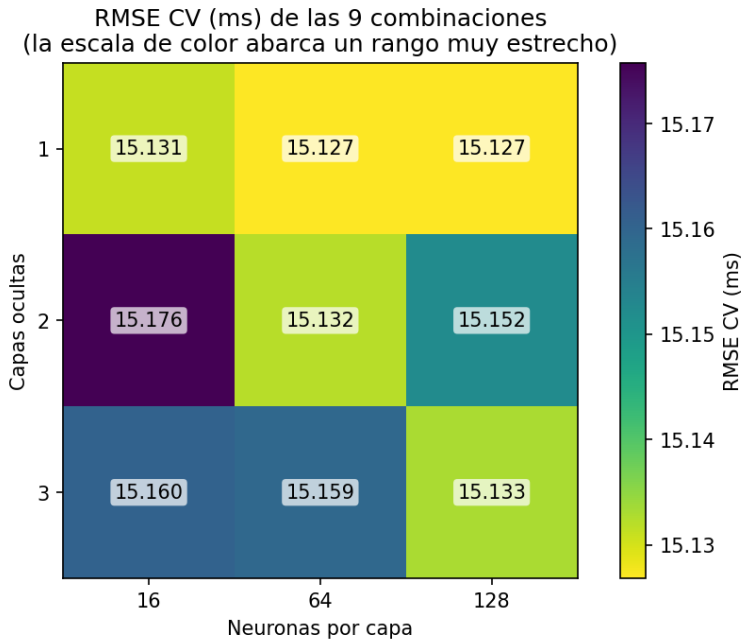


Figura 12. RMSE CV de las nueve combinaciones de capas y neuronas.

Comparación final de los modelos

Evaluación sobre el mismo conjunto TEST

La tabla 9 reúne los modelos de las dos entregas evaluados sobre el mismo conjunto TEST, ordenados de menor a mayor RMSE, y la figura 13 los compara gráficamente.

Modelo	**Entrega**	**MAE**	**MSE**	**RMSE**	**R^2**	**Reducción del RMSE frente al baseline (%)**
Red neuronal grande (128, 128, 128)	E2	11.9533	224.8669	14.9956	0.3784	21.16
Red neuronal suave (16)	E2	11.9592	224.8811	14.9960	0.3784	21.16
Red neuronal intermedia (64, 64)	E2	11.9553	224.9486	14.9983	0.3782	21.15
Regresión bayesiana	E2	12.0130	226.8301	15.0609	0.3730	20.82
SVR lineal (C=1, ε=5)	E2	12.0135	226.8563	15.0617	0.3729	20.81
Random Forest podado CV	E1	12.0862	229.5079	15.1495	0.3656	20.35
KNN K=50 (distance)	E1	12.1874	233.8547	15.2923	0.3536	19.60
Árbol podado CV	E1	12.2279	235.0786	15.3323	0.3502	19.39
Baseline (media)	—	15.1553	361.7885	19.0207	0.0000	0.00

Tabla 9. Comparación final de los modelos de ambas entregas sobre el mismo conjunto TEST (regresión).

Los cinco modelos de la segunda entrega obtienen un RMSE menor que los tres de la primera. Las tres redes neuronales quedan en los primeros lugares (RMSE entre 14.9956 y 14.9983 ms, R^2 de 0.378), seguidas por la regresión bayesiana y la SVR lineal (15.06 ms). Entre los modelos de la primera entrega se mantiene el orden ya conocido: Random Forest (15.1495 ms), KNN (15.2923 ms) y árbol podado (15.3323 ms). La red suave, elegida por validación cruzada, ocupa el segundo lugar. Las diferencias son pequeñas, 0.34 ms entre el mejor y el peor de los ocho modelos, y todos reducen el RMSE entre un 19 % y un 21 % frente al baseline.

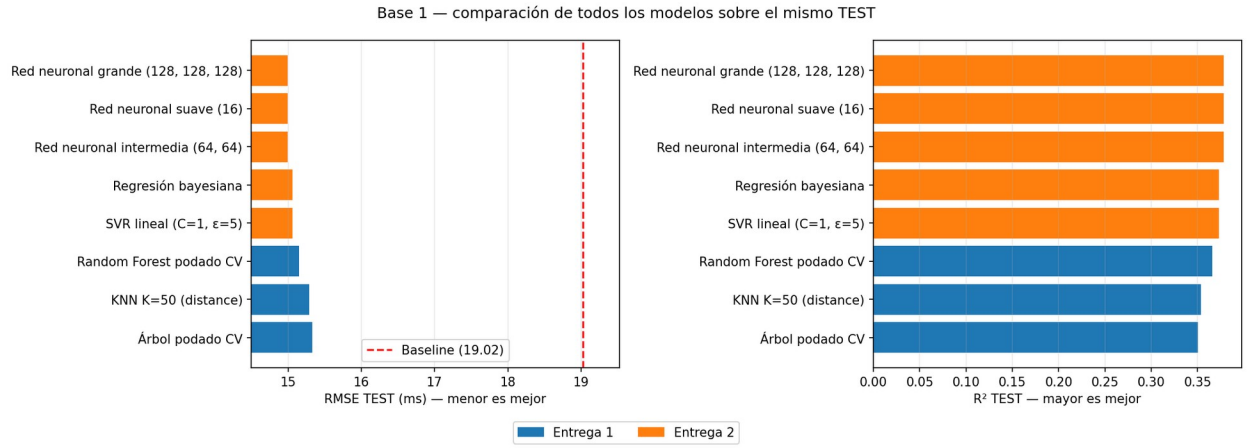


Figura 13. Comparación del RMSE y del R^2 de todos los modelos sobre TEST.

Comparación común mediante Cross-Validation

Para revisar si el orden depende de la partición de TEST, todos los modelos se evaluaron sobre la misma muestra de 8 000 registros de TRAIN y las mismas cinco particiones de la comparación común de la primera entrega (tabla 10). Los modelos de la primera entrega reproducen los valores reportados entonces.

Modelo	**RMSE CV, media ± desviación**	**MAE CV medio**	**R^2 CV medio**
Regresión bayesiana	15.0483 ± 0.3281	12.0093	0.3886
SVR lineal (C=1, ε=5)	15.0768 ± 0.3288	12.0268	0.3863
Red neuronal intermedia (64, 64)	15.1062 ± 0.3681	12.0312	0.3838
Red neuronal suave (16)	15.1326 ± 0.3340	12.0581	0.3817
Red neuronal grande (128, 128, 128)	15.1401 ± 0.3453	12.0612	0.3811
Random Forest podado CV	15.3361 ± 0.3344	12.2590	0.3650
KNN K=50 (distance)	15.6981 ± 0.2967	12.4916	0.3346
Árbol podado CV	15.8501 ± 0.3087	12.6504	0.3216
Baseline (media)	19.2487 ± 0.3250	15.3944	−0.0005

Tabla 10. Comparación común mediante Cross-Validation sobre 8 000 registros de TRAIN (regresión).

El resultado se mantiene: los cinco modelos nuevos obtienen menor RMSE que los tres de la primera entrega. Entre los modelos nuevos el orden cambia, porque en validación cruzada la regresión bayesiana queda primera y las redes al final, pero las diferencias entre ellos (menos de 0.1 ms) son mucho menores que la desviación entre folds (alrededor de 0.33 ms), así que en la práctica empatan. La ventaja sobre Random Forest, de 0.2 a 0.3 ms en validación cruzada y de 0.09 a 0.15 ms en TEST, también es menor que esa desviación, aunque se repite en las dos evaluaciones; por eso conviene leerla como una mejora pequeña y no como una diferencia contundente. Los valores de las redes no coinciden exactamente con los de la tabla 7 porque allí la validación cruzada se hizo sobre otra muestra, de 20 000 registros.

Costo computacional

La tabla 11 resume el costo de cada modelo, en el mismo orden de la tabla 9. La regresión bayesiana y la SVR lineal se entrenan en menos de un segundo con los 175 000 registros y obtienen prácticamente el mismo error que las redes neuronales, que tardan entre 3.4 y 31.2 segundos. Random Forest necesita 11.2 segundos, y KNN, que casi no tiene costo de entrenamiento, es con diferencia el más lento al predecir (37.9 segundos para los 75 000 registros de TEST), porque debe buscar los vecinos de cada observación. En este problema, los modelos lineales ofrecen la mejor relación entre precisión y costo.

Modelo	Entrega	Registros de entrenamiento	Entrenamiento (s)	Predicción de TEST (s)
Red neuronal grande (128, 128, 128)	E2	175 000	31.2	0.29
Red neuronal suave (16)	E2	175 000	3.41	0.05
Red neuronal intermedia (64, 64)	E2	175 000	11.6	0.12
Regresión bayesiana	E2	175 000	0.26	0.08
SVR lineal (C=1, ε=5)	E2	175 000	0.85	0.05
Random Forest podado CV	E1	175 000	11.2	0.19
KNN K=50 (distance)	E1	175 000	0.51	37.9
Árbol podado CV	E1	175 000	0.80	0.04

Tabla 11. Tiempo de entrenamiento y de predicción de cada modelo (regresión).

Datos reales frente a predicciones

La figura 14 compara el tiempo de reacción real de 4 000 jugadores de TEST con el predicho por cada modelo. Si un modelo fuera perfecto, todos los puntos caerían sobre la diagonal roja.

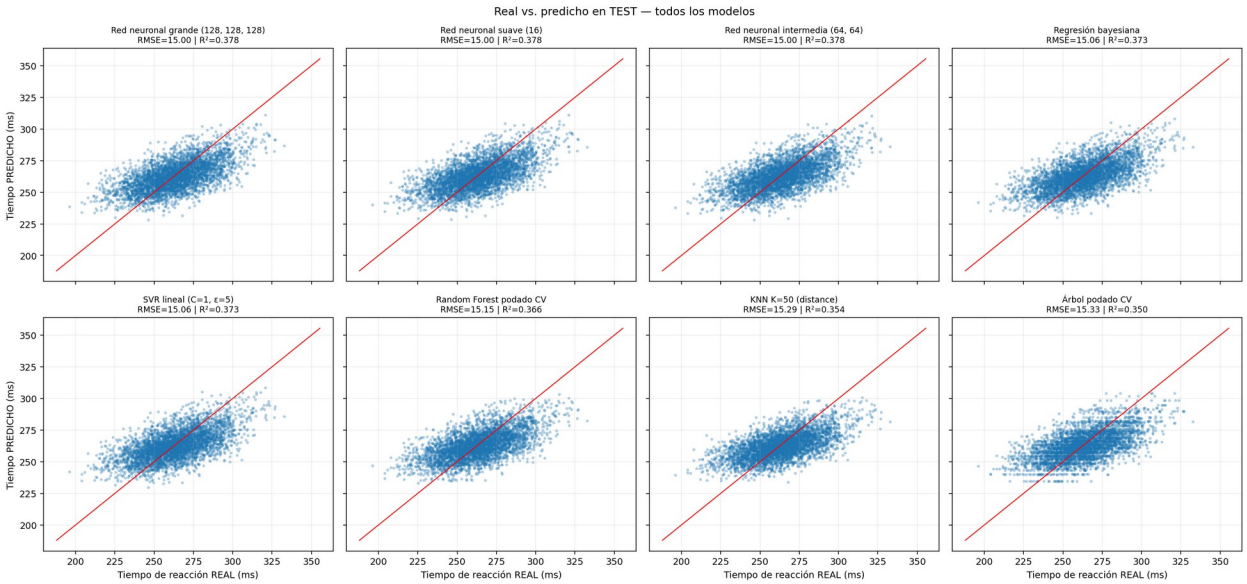


Figura 14. Valores reales frente a valores predichos en TEST para todos los modelos.

Todos los modelos concentran sus predicciones en un rango más estrecho que el real. En todo el conjunto TEST, los valores reales van de 187.9 a 355.5 ms, mientras que las predicciones de los modelos de la segunda entrega quedan entre 222.5 y 323.9 ms, y las de los modelos de la primera, entre 229.2 y 307.5 ms. Esta compresión hacia el centro es la forma visible de un R^2 moderado: con las variables disponibles, buena parte de la variación del tiempo de reacción no se puede explicar. El árbol de decisión forma, además, bandas horizontales, porque solo puede predecir un valor por hoja; los modelos lineales y las redes neuronales producen nubes continuas.

Clasificación

Punto de partida: modelos de la primera entrega

Al igual que en regresión, se volvieron a entrenar los tres clasificadores de la primera entrega con sus hiperparámetros y umbrales finales. La tabla 12 muestra que reproducen el F1 reportado. En el informe anterior, el ROC-AUC y el PR-AUC del árbol aparecen como 0.9127 y 0.8691 porque allí se truncaron; redondeados a cuatro decimales son 0.9128 y 0.8692, que son los valores usados en este informe.

Modelo	Umbral	F1 TEST	ROC-AUC TEST	F1 reportado en la Entrega 1	¿Coincide?
Random Forest ajustado	0.49	0.7955	0.9288	0.7955	Sí
Árbol podado	0.58	0.7708	0.9128	0.7708	Sí
KNN ajustado	0.43	0.7592	0.9014	0.7592	Sí

Tabla 12. Verificación de los modelos de la primera entrega sobre TEST (clasificación).

Resultados de Naive Bayes

Antes de entrenar Naive Bayes se midió la dependencia entre atributos dentro de cada clase, justamente lo que el método supone nulo. La figura 15 y la tabla 13 muestran que varios pares están claramente relacionados aun conociendo la clase: el tipo de hotel con la tarifa ($V = 0.351$), el segmento de mercado con las solicitudes especiales (0.332) y con el tipo de

cliente (0.319), entre otros. La suposición de independencia, por lo tanto, no se cumple en estos datos.

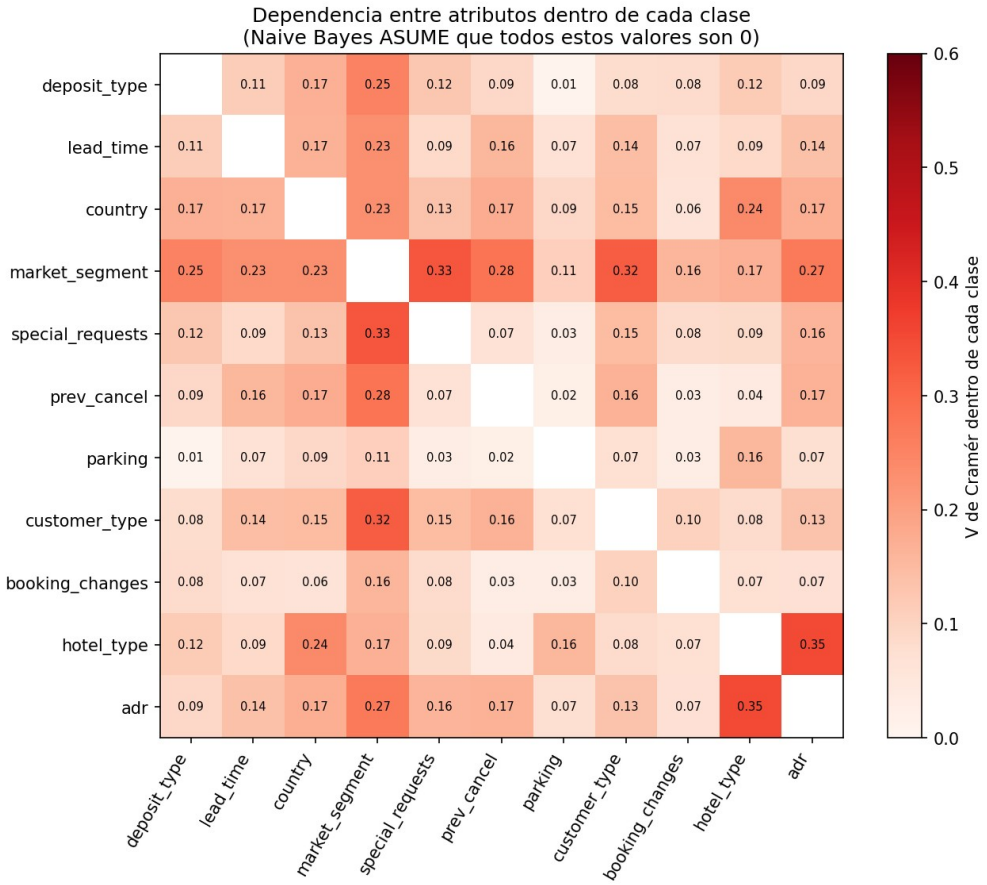


Figura 15. Dependencia entre los once atributos discretizados dentro de cada clase (V de Cramér).

Atributo A	**Atributo B**	**V de Cramér dentro de cada clase**
hotel_type	adr	0.351
market_segment	special_requests	0.332
market_segment	customer_type	0.319
market_segment	prev_cancel	0.279
market_segment	adr	0.268
deposit_type	market_segment	0.252
country	hotel_type	0.241
country	market_segment	0.229

Tabla 13. Pares de atributos más dependientes entre sí dentro de cada clase.

La tabla 14 compara las variantes evaluadas. CategoricalNB con 10 intervalos y alpha = 0.5 obtuvo el mayor ROC-AUC de validación cruzada (0.8315), por encima de cualquier configuración de GaussianNB (máximo 0.8090), y fue la variante elegida. Con el umbral de 0.5, la variante con mayor F1 fue GaussianNB con var_smoothing = 1e-06 (0.6543), pero su ROC-AUC es más bajo (0.8017). Los valores elegidos (10 intervalos y alpha = 0.5) quedan en el borde de los probados; más intervalos podrían mejorar un poco a CategoricalNB, aunque la ganancia al pasar de 5 a 10 fue de solo 0.005 de ROC-AUC.

Variante	Parámetros	F1 CV*	Recall CV*	Precision CV*	Balanced Acc. CV*	ROC-AUC CV
CategoricalNB	n_bins = 10, alpha = 0.5	0.6436	0.5383	0.8002	0.7296	0.8315
CategoricalNB	n_bins = 10, alpha = 1.0	0.6426	0.5360	0.8022	0.7291	0.8314
CategoricalNB	n_bins = 5, alpha = 0.5	0.6323	0.5226	0.8001	0.7229	0.8260
CategoricalNB	n_bins = 5, alpha = 1.0	0.6313	0.5201	0.8028	0.7225	0.8260
GaussianNB	var_smoothing = 0.1	0.6201	0.8799	0.4787	0.6580	0.8090
GaussianNB	var_smoothing = 0.001	0.6421	0.7966	0.5379	0.6969	0.8024
GaussianNB	var_smoothing = 1e-09	0.6000	0.5063	0.7824	0.7034	0.8017
GaussianNB	var_smoothing = 1e-06	0.6543	0.7304	0.5951	0.7171	0.8017

Tabla 14. Variantes de Naive Bayes evaluadas mediante validación cruzada sobre todo TRAIN (* con el umbral por defecto).

El umbral elegido con las predicciones out-of-fold fue 0.28, bastante menor que 0.5. La figura 16, calculada sobre TEST solo con fines ilustrativos, ayuda a entenderlo: las reservas no canceladas reciben en su mayoría probabilidades bajas, concentradas cerca de 0, mientras que las canceladas se reparten de forma casi uniforme entre 0 y 1, con una acumulación cerca de 1. Como muchas cancelaciones reciben probabilidades bajas, el umbral tiene que bajar para detectarlas. Una explicación probable es que, al multiplicar como si fueran independientes atributos que repiten información, el modelo cuenta dos veces la misma evidencia y sus probabilidades quedan mal calibradas. Los resultados en TEST se presentan en la tabla 15.

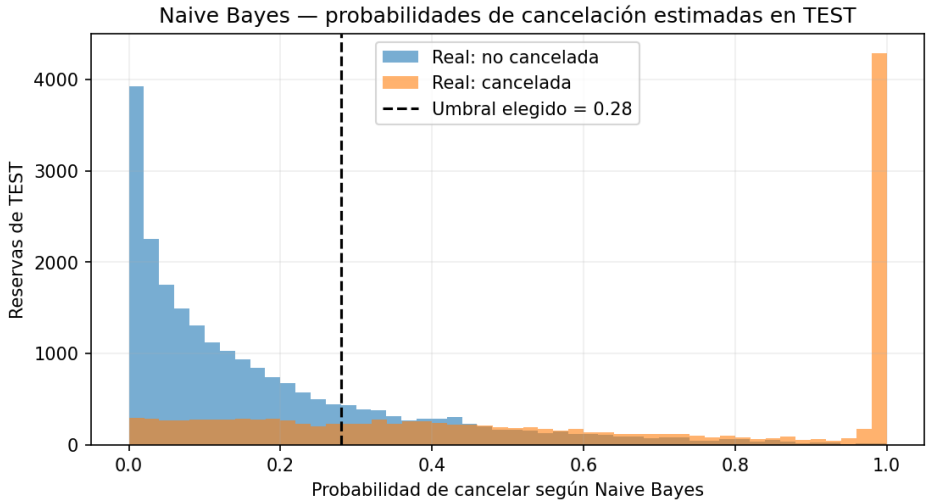


Figura 16. Distribución de las probabilidades de cancelación estimadas por Naive Bayes en TEST.

Métrica	**Resultado**
Umbral	0.28
Accuracy	0.7579
Balanced Accuracy	0.7505
Precision	0.6579
Recall	0.7219
F1-score	0.6884
ROC-AUC	0.8308
PR-AUC	0.7957

Tabla 15. Resultados TEST de Naive Bayes categórico.

Con un F1 de 0.6884 y un ROC-AUC de 0.8308, Naive Bayes es el clasificador con menor desempeño de las dos entregas, aunque supera ampliamente al baseline. A cambio, es uno de los modelos más rápidos: se entrena en menos de medio segundo.

Resultados de la Red Bayesiana

La Red Bayesiana aprendió 30 flechas entre las doce variables, los once atributos y la clase (figura 17). La clase tiene como padres el tipo de depósito, el segmento de mercado y el país, y como hijos la anticipación de la reserva, las solicitudes especiales, el estacionamiento, los

cambios en la reserva y las cancelaciones previas. Su manto de Markov, formado por los padres, los hijos y los otros padres de los hijos, incluye además el tipo de cliente y el tipo de hotel, de modo que contiene diez de los once atributos. Solo la tarifa (adr) queda fuera: según la red, una vez conocidos los demás atributos, la tarifa no aporta información adicional sobre la cancelación.

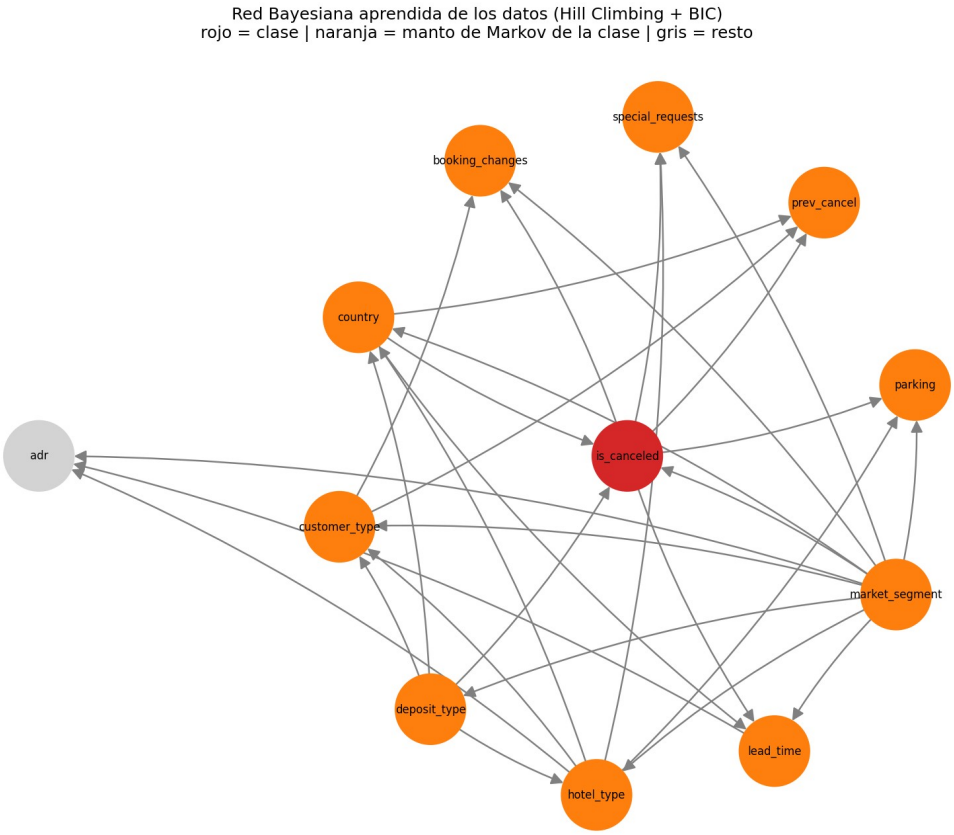


Figura 17. Red Bayesiana aprendida a partir de los datos.

La dirección de las flechas no debe leerse como causa y efecto. La puntuación BIC asigna el mismo valor a grafos equivalentes (con el mismo esqueleto y las mismas estructuras en V), de modo que los datos no determinan la orientación de muchas flechas, y lo que la búsqueda encuentra son dependencias estadísticas. Por ejemplo, la flecha de is_canceled hacia lead_time no significa que cancelar modifique la anticipación de la reserva, sino que en estos datos ambas variables están relacionadas.

A diferencia de los demás modelos, la Red Bayesiana permite consultar cómo cambia la probabilidad de cancelar al conocer cierta información, mediante inferencia probabilística. En la tabla 16, la probabilidad base de 37.0 % sube a 99.4 % cuando el depósito es no reembolsable y a 91.9 % cuando el cliente tiene cancelaciones previas, y baja a 0.05 % cuando la reserva solicitó estacionamiento. Estas probabilidades describen patrones presentes en el dataset.

Evidencia conocida sobre la reserva	**P(cancelar)**
Sin información (probabilidad base)	37.0 %
Depósito no reembolsable	99.4 %
Sin depósito	28.3 %
Reservó con más de 180 días de anticipación	56.9 %
Pidió estacionamiento	0.05 %
Hizo 2 o más solicitudes especiales	20.8 %
Canceló antes otra reserva	91.9 %
Agencia online + Portugal	45.4 %

Tabla 16. Consultas de probabilidad sobre la Red Bayesiana.

La tabla 17 compara la Red Bayesiana con el Naive Bayes entrenado sobre los mismos once atributos. La diferencia principal entre ambos es la estructura, supuesta en un caso y aprendida en el otro. También cambia el suavizado de las probabilidades (alpha = 1 en Naive Bayes y prior BDeu en la red), aunque con más de 80 000 registros de entrenamiento ese efecto debería ser menor.

Modelo (mismos 11 atributos)	**Estructura**	**Umbral**	**F1**	**Recall**	**Precision**	**Balanced Acc.**	**ROC-AUC**
Naive Bayes	Fija: independencia supuesta	0.27	0.7011	0.7293	0.6749	0.7613	0.8495
Red Bayesiana	Aprendida: 30 dependencias	0.41	0.7531	0.7485	0.7578	0.8039	0.8993

Tabla 17. Naive Bayes frente a Red Bayesiana con los mismos once atributos (TEST).

Modelar las dependencias, en lugar de suponer independencia, sube el F1 de 0.7011 a 0.7531 y el ROC-AUC de 0.8495 a 0.8993. Esta diferencia da una medida del costo de la suposición de Naive Bayes en este problema. Con solo once atributos discretizados, el ROC-AUC de la Red Bayesiana queda cerca del de KNN (0.9014), que usa los 28 atributos.

Resultados de la máquina de soporte vectorial

La tabla 18 presenta las doce configuraciones evaluadas. El kernel RBF con C = 1 y class_weight = 'balanced' obtuvo el mayor F1 de validación cruzada (0.7659), frente a 0.7375 del mejor kernel lineal; en ROC-AUC la ventaja es menor (0.8941 frente a 0.8824). A diferencia de la regresión, aquí una frontera no lineal mejora el resultado, aunque de forma moderada. Esa misma configuración tiene también el mayor ROC-AUC de validación cruzada, así que la elección no depende de haber usado F1 con el umbral por defecto. También se observa el efecto de class_weight = 'balanced': con el mismo kernel y el mismo C, aumenta el Recall a cambio de algo de Precision.

Kernel	**C**	**class_weight**	**F1 CV***	**Desv. F1 CV**	**Recall CV***	**Precision CV***	**ROC-AUC CV**
RBF	1	balanced	0.7659	0.0116	0.7746	0.7578	0.8941
RBF	10	None	0.7591	0.0080	0.7216	0.8009	0.8901
RBF	10	balanced	0.7525	0.0082	0.7692	0.7367	0.8903
RBF	1	None	0.7417	0.0157	0.6564	0.8524	0.8911
lineal	0.01	balanced	0.7375	0.0102	0.7567	0.7195	0.8824
lineal	0.1	balanced	0.7375	0.0097	0.7584	0.7179	0.8822
lineal	1	balanced	0.7373	0.0095	0.7597	0.7164	0.8819
RBF	0.1	balanced	0.7248	0.0127	0.7033	0.7485	0.8708
lineal	0.1	None	0.7129	0.0128	0.6277	0.8252	0.8818
lineal	1	None	0.7128	0.0129	0.6284	0.8238	0.8815
lineal	0.01	None	0.7120	0.0139	0.6217	0.8333	0.8821
RBF	0.1	None	0.6058	0.0203	0.4499	0.9282	0.8750

Tabla 18. Configuraciones de la SVM evaluadas mediante validación cruzada sobre 8 000 registros de TRAIN (* con el umbral por defecto: puntuación ≥ 0).

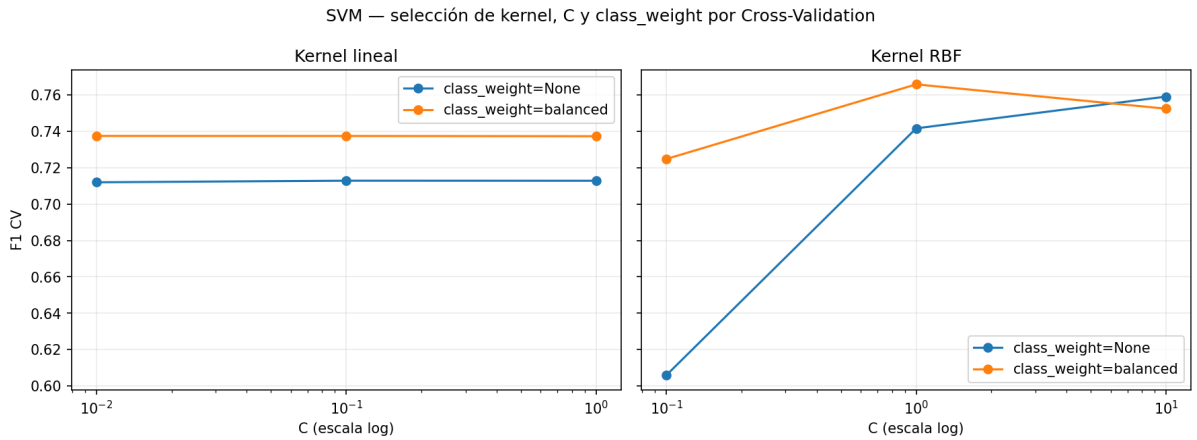


Figura 18. Selección de kernel, C y class_weight de la SVM mediante validación cruzada.

El modelo final se entrenó con 30 000 registros de TRAIN, de los cuales 13 710 (el 46 %) quedaron como vectores de soporte. Una proporción tan alta refleja el solapamiento entre las clases y el margen suave con C = 1, y explica que la SVM sea lenta para predecir: cada predicción compara la reserva con todos esos vectores. Los resultados en TEST se presentan en la tabla 19; el umbral está expresado en la escala de la puntuación de la SVM.

Métrica	**Resultado**
Umbral sobre la puntuación	+0.041
Accuracy	0.8310
Balanced Accuracy	0.8208
Precision	0.7666
Recall	0.7817
F1-score	0.7741
ROC-AUC	0.9056
PR-AUC	0.8715

Tabla 19. Resultados TEST de la SVM con kernel RBF.

Resultados de las tres redes neuronales

La tabla 20 resume las tres redes de clasificación. En validación cruzada obtienen prácticamente el mismo F1 (entre 0.7507 y 0.7517) y el mismo ROC-AUC (entre 0.9066 y 0.9076); las diferencias son de una milésima, muy por debajo de la variación entre folds (de

± 0.0044 a ± 0.0138). Según el criterio definido en la metodología, las tres quedan igual y se elige la red suave por ser la más simple.

Red	**Conexiones**	**F1 CV* (± desv.)**	**ROC-AUC CV**	**Umbral**	**F1 TEST**	**ROC-AUC TEST**	**Respecto a la red anterior (F1 CV)**
Suave (16)	1 600	0.7517 ± 0.0111	0.9069	0.38	0.7779	0.9173	Punto de partida
Intermedia (64, 64)	10 496	0.7507 ± 0.0044	0.9066	0.38	0.7870	0.9212	Quedó igual (−0.0010)
Grande (128, 128, 128)	45 568	0.7510 ± 0.0138	0.9076	0.34	0.7815	0.9192	Quedó igual (+0.0003)

Tabla 20. Comparación de las tres redes neuronales de clasificación (CV sobre 20 000 registros de TRAIN, * con umbral 0.5; TEST con la red entrenada en todo TRAIN y el umbral ajustado).

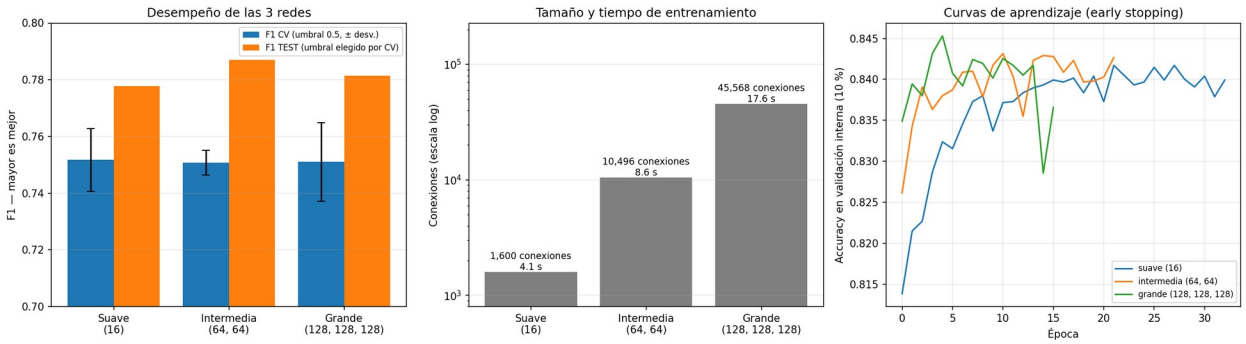


Figura 19. Desempeño, tamaño, tiempo de entrenamiento y curvas de aprendizaje de las tres redes de clasificación.

En TEST, la red intermedia obtuvo el mejor F1 (0.7870), frente a 0.7779 de la suave y 0.7815 de la grande: pasar de la red suave a la intermedia sube el F1 en 0.009, y pasar a la grande lo baja en 0.006. Estas diferencias no bastan para afirmar que una red más grande sea mejor. Tampoco se puede descartar que, con todo TRAIN, la red intermedia aproveche mejor los datos que en la validación cruzada, que usó 20 000 registros; pero como TEST no se usa para elegir, se mantiene la red suave. El F1 de TEST es, además, mayor que el F1 CV* en las tres redes, porque en TEST se usa el umbral ajustado y redes entrenadas con todo TRAIN. Mientras tanto, las conexiones pasan de 1 600 a 45 568 y el tiempo de entrenamiento, de 4.1 a 17.6 segundos.

Las nueve combinaciones (tabla 21 y figura 20) quedan entre 0.7451 y 0.7574 de F1 CV. La mejor fue una sola capa de 128 neuronas, pero su ventaja sobre la red suave (0.006) también cae dentro de la variación entre folds, y ni agregar capas ni aumentar neuronas cambia el resultado de forma consistente.

Arquitectura	**Conexiones**	**F1 CV***	**Al agregar una capa**	**Al aumentar neuronas**
(16)	1 600	0.7517 ± 0.0111	—	—
(64)	6 400	0.7538 ± 0.0126	—	Quedó igual (+0.0021)
(128)	12 800	0.7574 ± 0.0082	—	Quedó igual (+0.0036)
(16, 16)	1 856	0.7504 ± 0.0106	Quedó igual (−0.0014)	—
(64, 64)	10 496	0.7507 ± 0.0044	Quedó igual (−0.0030)	Quedó igual (+0.0004)
(128, 128)	29 184	0.7480 ± 0.0131	Quedó igual (−0.0094)	Quedó igual (−0.0027)
(16, 16, 16)	2 112	0.7546 ± 0.0065	Quedó igual (+0.0042)	—
(64, 64, 64)	14 592	0.7451 ± 0.0073	Quedó igual (−0.0056)	Quedó igual (−0.0094)
(128, 128, 128)	45 568	0.7510 ± 0.0138	Quedó igual (+0.0030)	Quedó igual (+0.0059)

Tabla 21. Efecto de agregar capas y neuronas en las nueve combinaciones (clasificación; * con el umbral por defecto).

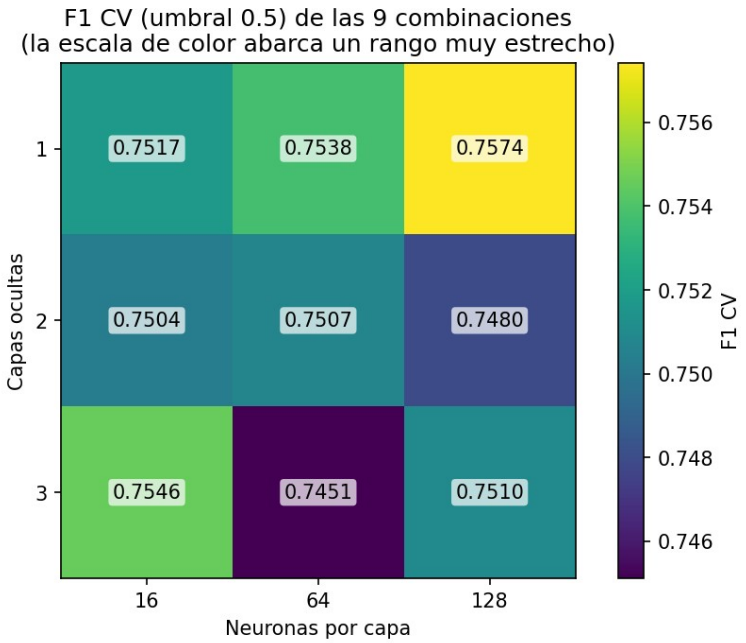


Figura 20. F1 CV de las nueve combinaciones de capas y neuronas.

Comparación final de modelos

Los modelos de las dos entregas se evaluaron sobre el mismo conjunto TEST, cada uno con el umbral elegido por validación cruzada. La tabla 22 los presenta ordenados por F1 y la figura 21 los compara gráficamente.

Modelo	Entrega	Accuracy	Balanced Acc.	Precision	Recall	F1	ROC-AUC	PR-AUC
Random Forest ajustado	E1	0.8463	0.8382	0.7842	0.8071	0.7955	0.9288	0.8986
Red neuronal intermedia (64, 64)	E2	0.8397	0.8314	0.7748	0.7997	0.7870	0.9212	0.8901
Red neuronal grande (128, 128, 128)	E2	0.8341	0.8272	0.7632	0.8006	0.7815	0.9192	0.8879
Red neuronal suave (16)	E2	0.8277	0.8250	0.7445	0.8144	0.7779	0.9173	0.8853
SVM RBF (C=1)	E2	0.8310	0.8208	0.7666	0.7817	0.7741	0.9056	0.8715
Árbol podado	E1	0.8362	0.8172	0.7999	0.7438	0.7708	0.9128	0.8692
KNN ajustado	E1	0.8235	0.8086	0.7674	0.7512	0.7592	0.9014	0.8696
Red Bayesiana (11 atributos)	E2	0.8182	0.8039	0.7578	0.7485	0.7531	0.8993	0.8617
Naive Bayes Categórico (10 intervalos)	E2	0.7579	0.7505	0.6579	0.7219	0.6884	0.8308	0.7957
Baseline (clase mayoritaria)	—	0.6296	0.5000	0.0000	0.0000	0.0000	0.5000	0.3704

Tabla 22. Comparación final de los modelos de ambas entregas sobre el mismo conjunto TEST (clasificación).

Random Forest, de la primera entrega, sigue siendo el mejor modelo, con un F1 de 0.7955 y un ROC-AUC de 0.9288. Le siguen las tres redes neuronales (F1 entre 0.7779 y 0.7870) y la SVM RBF (0.7741), que en F1 y en Balanced Accuracy superan al árbol podado (0.7708) y a KNN (0.7592). En ROC-AUC el orden cambia en parte, porque el árbol (0.9128) queda por encima de la SVM (0.9056). La red suave, que es la elegida por validación cruzada, ocupa el cuarto lugar. La Red Bayesiana (0.7531) queda por debajo de KNN, y Naive Bayes (0.6884) ocupa el último lugar. Todos superan ampliamente al baseline, cuyo F1 es 0 porque nunca predice una cancelación.

Al leer esta tabla conviene tener presentes algunas diferencias de condiciones: la SVM se entrenó con 30 000 registros y no con los 83 573 de TRAIN, la Red Bayesiana usa solo once

atributos discretizados y las redes neuronales conservan hiperparámetros por defecto, mientras que Random Forest fue ajustado en la primera entrega. Estas diferencias pueden explicar parte de las distancias entre modelos.

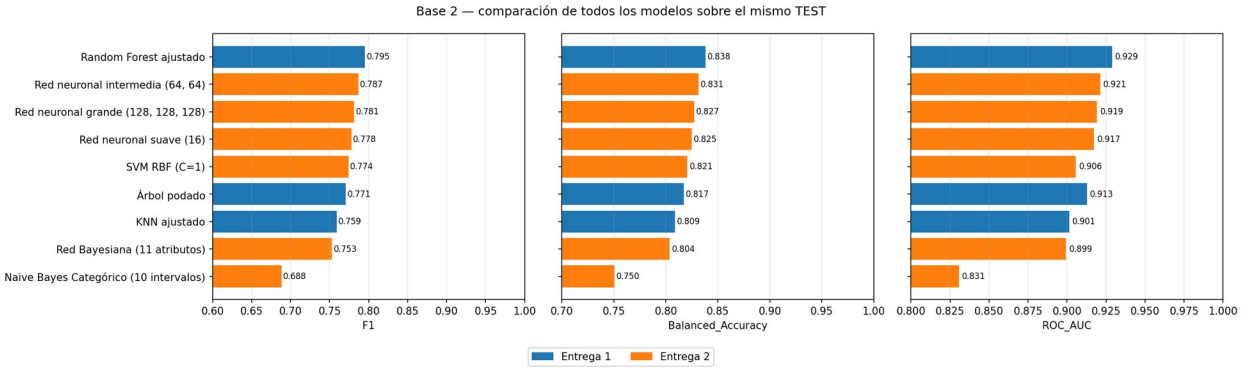


Figura 21. Comparación final del F1, la Balanced Accuracy y el ROC-AUC de todos los modelos sobre TEST.

Las curvas ROC y Precision-Recall de la figura 22, que no dependen del umbral, confirman los extremos del ordenamiento: Random Forest y las redes neuronales tienen las curvas más altas y Naive Bayes la más baja. En la zona intermedia los modelos están muy próximos; por ejemplo, el PR-AUC de KNN (0.8696) y el del árbol (0.8692) son prácticamente iguales.

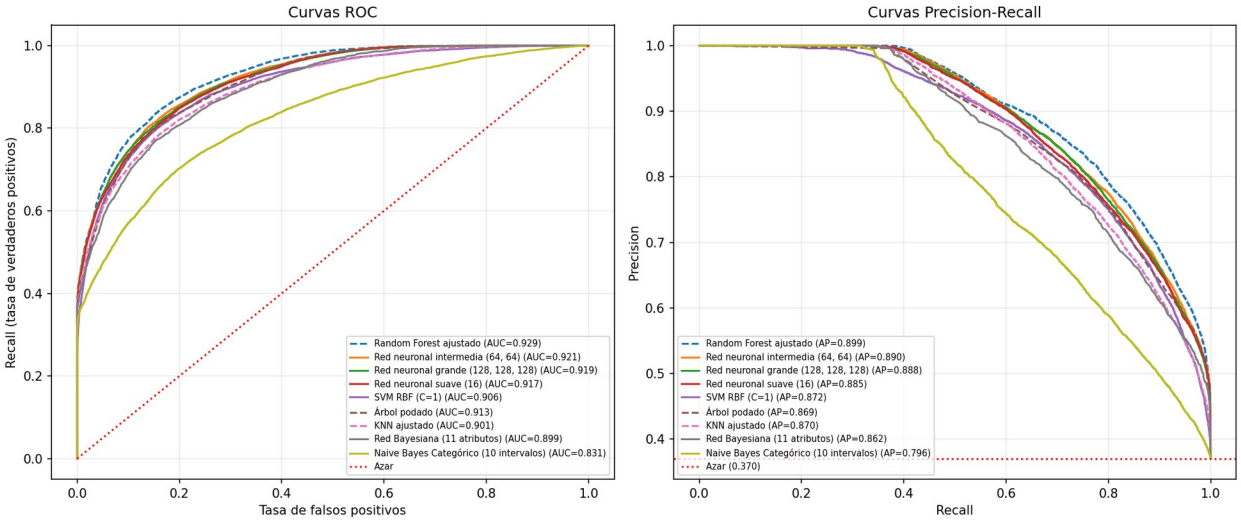


Figura 22. Curvas ROC y Precision-Recall de todos los modelos sobre TEST.

En costo computacional también hay diferencias grandes (tabla 23, en el mismo orden de la tabla 22). Random Forest, con 500 árboles, tarda 69.3 segundos en entrenarse, mientras que las redes neuronales necesitan entre 4.1 y 17.6 segundos y predicen TEST en menos de 0.2 segundos. La SVM y KNN son los más costosos en conjunto, con unos 89 segundos cada uno: la SVM necesita 46.1 segundos para entrenarse con solo 30 000 registros y 43.0 para predecir TEST, porque cada predicción se compara con sus 13 710 vectores de soporte, mientras que KNN casi no tiene costo de entrenamiento pero tarda 89.0 segundos en predecir. Naive Bayes, el árbol y la Red Bayesiana necesitan, cada uno, menos de 2.5 segundos entre entrenamiento y predicción.

Modelo	**Entrega**	**Registros de entrenamiento**	**Entrenamiento (s)**	**Predicción de TEST (s)**
Random Forest ajustado	E1	83 573	69.3	0.94
Red neuronal intermedia (64, 64)	E2	83 573	8.56	0.11
Red neuronal grande (128, 128, 128)	E2	83 573	17.6	0.18
Red neuronal suave (16)	E2	83 573	4.09	0.11
SVM RBF (C=1)	E2	30 000	46.1	43.0
Árbol podado	E1	83 573	1.22	0.10
KNN ajustado	E1	83 573	0.41	89.0
Red Bayesiana (11 atributos)	E2	83 573	2.05	0.17
Naive Bayes Categórico (10 intervalos)	E2	83 573	0.30	0.08

Tabla 23. Tiempo de entrenamiento y de predicción de cada modelo (clasificación).

Finalmente, la figura 23 presenta las matrices de confusión de los modelos de la segunda entrega con sus umbrales seleccionados.

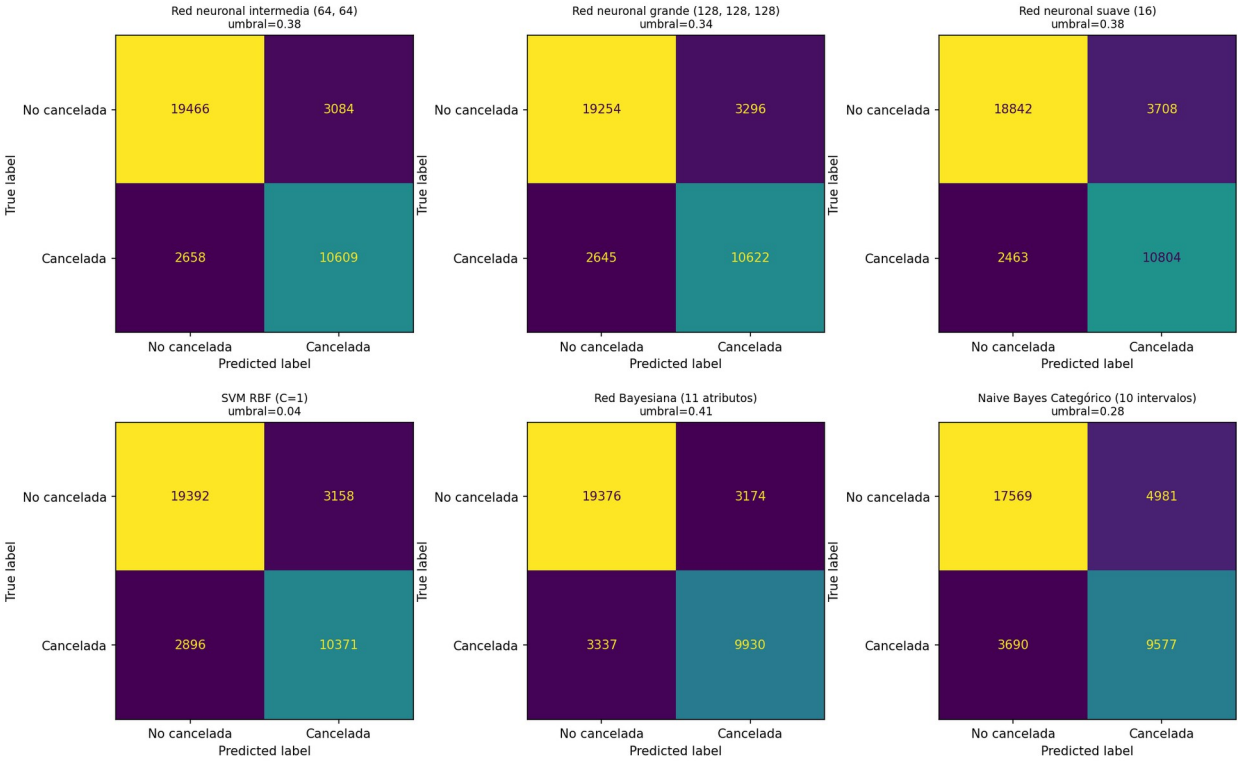


Figura 23. Matrices de confusión de los modelos de la segunda entrega sobre TEST.

CONCLUSIÓN

En esta entrega se incorporaron modelos bayesianos, máquinas de soporte vectorial y redes neuronales, y se compararon con los modelos de la primera entrega usando los mismos datos, la misma partición y el mismo conjunto TEST. Haber reproducido primero las métricas anteriores permitió que la comparación partiera de una base común.

En regresión, los cinco modelos nuevos obtuvieron un RMSE menor que los de la primera entrega, tanto en TEST como en la validación cruzada común. Las redes neuronales lograron el menor error en TEST (entre 14.9956 y 14.9983 ms, con un R^2 de 0.378), seguidas muy de cerca por la regresión bayesiana y la SVR lineal (15.06 ms), mientras que Random Forest, el mejor modelo de la primera entrega, obtuvo 15.1495 ms. Las diferencias son pequeñas pero consistentes, y los resultados apuntan a una relación aproximadamente lineal: la SVR prefirió el kernel lineal y las redes no superaron de forma apreciable a la regresión bayesiana, que es un modelo lineal. Esta última, además, se puede interpretar: más horas de sueño se asocian con un tiempo de reacción menor, y más horas de juego y una mayor edad, con uno mayor. Ningún modelo pasó de un R^2 de 0.38, lo que sugiere que el límite está en la información que contienen las variables más que en el algoritmo.

En clasificación ocurrió lo contrario: una frontera no lineal ayudó. La SVM con kernel RBF superó a la lineal, y los modelos que capturan interacciones entre atributos, Random Forest y las redes neuronales, obtuvieron los mejores resultados. Random Forest se mantuvo en el primer lugar (F1 de 0.7955), seguido de cerca por las redes neuronales, que se entrenan en bastante menos tiempo. La SVM RBF superó en F1 al árbol y a KNN, pero es costosa de entrenar y lenta para predecir.

Naive Bayes fue el modelo más débil. Las mediciones mostraron que su suposición de independencia no se cumple, y la Red Bayesiana, que aprende las dependencias en lugar de suponerlas, subió el F1 de 0.7011 a 0.7531 con los mismos atributos. La Red Bayesiana fue, además, uno de los modelos más interpretables: muestra con qué variables se relaciona la cancelación y permite calcular probabilidades para situaciones concretas.

Sobre las tres redes neuronales, en los dos problemas aumentar neuronas y capas ocultas no mejoró el modelo de forma apreciable. En regresión las tres obtuvieron el mismo error, aunque las conexiones se multiplicaron por más de 150 y el tiempo de entrenamiento, por cerca de nueve. En clasificación empataron en validación cruzada, y las diferencias que aparecieron en TEST fueron pequeñas y no bastan para preferir una red más grande. En ambos problemas la red suave resultó la elección más razonable. A diferencia del árbol sin poda de la primera entrega, que empeoró al crecer, aquí la complejidad adicional no perjudicó, pero tampoco aportó.

En conjunto, los resultados indican que el mejor modelo depende de la estructura del problema. Cuando la relación es aproximadamente lineal, un modelo sencillo como la regresión bayesiana rinde igual que alternativas mucho más costosas y, además, explica sus predicciones; cuando hay interacciones entre atributos, los ensambles y las redes neuronales obtienen ventaja. Los modelos incorporados en esta entrega no reemplazaron a Random Forest en clasificación, pero sí aportaron mejoras pequeñas en regresión (entre 0.09 y 0.15 ms de RMSE en TEST frente a Random Forest) y herramientas nuevas para entender los datos, como la incertidumbre de las predicciones y el grafo de dependencias.